

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP
KHOA ĐIỆN TỬ



TIỂU LUẬN KIỂM THỬ PHẦN MỀM
ĐỀ TÀI: NỀN TẢNG AI/ML CẦN BIẾT CHO KIỂM
THỬ PHẦN MỀM HIỆN ĐẠI

GVHD : TS. NGUYỄN TUẤN LINH
HỌ VÀ TÊN SINH VIÊN : HÀU THỊ THANH HUYỀN
MSSV : K225480106027
Lớp : K58.KTP

THÁI NGUYỄN - 2026

TRƯỜNG ĐHKTCN CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

KHOA ĐIỆN TỬ

Độc lập - Tự do - Hạnh phúc

PHIẾU GIAO NHIỆM VỤ

Sinh viên 1: HẦU THỊ THANH HUYỀN

MSSV: K225480106027

Lớp: K58KTP

Khoá: 2022-2027

Bộ môn: Công Nghệ Thông Tin

Giáo viên hướng dẫn: TS. NGUYỄN TUẤN LINH

1. Tên đề án: Nền tảng AI/ML cần biết cho kiểm thử phần mềm hiện đại

2. Nội dung thực hiện :

- Tóm tắt các loại AI/ML
- Học có giám sát/không giám sát và khả năng áp dụng trong QA.
- Chương trình mô phỏng

5. Ngày giao nhiệm vụ: 03/06/2026

6. Ngày hoàn thành nhiệm vụ: 24/06/2026

SINH VIÊN

(Ký và ghi rõ họ tên)

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày....tháng.....năm 20....

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

MỤC LỤC

MỤC LỤC.....	3
DANH MỤC CÁC TỪ VIẾT TẮT	6
DANH MỤC CÁC BẢNG VÀ HÌNH VẼ, ĐỒ THỊ.....	7
LỜI CAM ĐOAN	1
LỜI NÓI ĐẦU	2
CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI	3
1.1. Giới thiệu môn học.....	3
1.2. Lý do chọn đề tài.....	4
1.3. Mục tiêu nghiên cứu.....	5
1.4. Phạm vi nghiên cứu.....	6
1.5. Phương pháp nghiên cứu.....	6
1.5.1. Nghiên cứu tài liệu	6
1.5.2. Thu thập và xây dựng dữ liệu	7
1.5.3. Xây dựng mô hình Machine Learning	7
1.5.4. Đánh giá kết quả.....	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	9
2.1. Trí tuệ nhân tạo trong kiểm thử phần mềm.....	9
2.1.1. Đặc điểm của AI.....	9
2.1.2. Vai trò của AI trong kiểm thử phần mềm	10
2.2. Học máy (Machine Learning - ML).....	10
2.2.1. Quy trình xây dựng mô hình Machine Learning	11
2.3. Học có giám sát (Supervised Learning).....	12
2.3.1. Nguyên lý hoạt động	12
2.3.2. Ứng dụng trong đề tài	13
2.3.3. Ưu điểm.....	13
2.3.4. Nhược điểm.....	14

2.4. Học không giám sát (Unsupervised Learning)	14
2.4.1. Nguyên lý hoạt động	14
2.4.2. Ứng dụng trong QA	15
2.4.3. Ưu điểm.....	15
2.4.4. Nhược điểm.....	16
2.5. Học tăng cường (Reinforcement Learning)	16
2.5.1. Liên hệ với kiểm thử phần mềm	16
2.5.2. Ưu điểm và giới hạn.....	17
2.6. Thuật toán Decision Tree	17
2.6.1. Cơ chế hoạt động của Decision Tree	17
2.6.2. Lý do lựa chọn Decision Tree trong đề tài	18
2.6.3. Ưu điểm của Decision Tree	18
2.6.4. Nhược điểm của Decision Tree.....	18
2.7. Ứng dụng AI/ML trong kiểm thử phần mềm.....	19
2.7.1. Tự động sinh test case	19
2.7.2. Phân loại lỗi phần mềm.....	19
2.7.3. Dự đoán lỗi.....	19
2.7.4. Phân tích log.....	19
2.7.5. Kiểm thử giao diện.....	20
2.8. Liên hệ với chất lượng phần mềm	20
CHƯƠNG 3. PHÂN TÍCH BÀI TOÁN VÀ XÂY DỰNG MÔ HÌNH	21
3.1. Giới thiệu bài toán.....	21
3.2. Mô tả bài toán.....	22
3.2.1. Dữ liệu đầu vào	22
3.3. Bộ dữ liệu sử dụng	23
3.4. Phân tích dữ liệu.....	25
3.4.1. Thuộc tính Bug_Count.....	25
3.4.2. Thuộc tính Crash	25
3.4.3. Thuộc tính Severity	25

3.5. Lựa chọn thuật toán.....	26
3.6. Xây dựng mô hình Machine Learning	27
3.7. Quy trình hoạt động của hệ thống.....	29
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ	31
4.1. Môi trường thực nghiệm	31
4.2. Chuẩn bị dữ liệu thực nghiệm.....	33
4.3. Tách dữ liệu đầu vào và đầu ra	35
4.4. Chia dữ liệu huấn luyện và kiểm tra	35
4.5. Huấn luyện mô hình Decision Tree	36
4.6. Đánh giá độ chính xác của mô hình.....	37
4.7. Thử nghiệm dự đoán lỗi mới.....	38
4.8. Đánh giá kết quả thực nghiệm	40
4.8.1. Kết quả đạt được	40
4.8.2. Ưu điểm của mô hình.....	40
4.8.3. Hạn chế của mô hình.....	41
4.8.4. Khả năng ứng dụng trong kiểm thử phần mềm	41
TÀI LIỆU THAM KHẢO.....	43

DANH MỤC CÁC TỪ VIẾT TẮT

1. AI :Trí tuệ nhân tạo
2. ML :Học máy
3. QA: Đảm bảo chất lượng phần mềm
4. CSV :Tập dữ liệu phân tách bằng dấu phẩy
5. VS Code :Môi trường phát triển mã nguồn
6. API :Giao diện lập trình ứng dụng
7. GUI: Giao diện người dùng đồ họa
8. IT :Công nghệ thông tin

DANH MỤC CÁC BẢNG VÀ HÌNH VẼ, ĐỒ THỊ

STT	Tên hình
------------	-----------------

Hình 3.2	Nội dung file bugs.csv
----------	------------------------

Hình 3.3	Minh họa các mức độ lỗi phần mềm
----------	----------------------------------

Hình 3.5	Mô hình cây quyết định (Decision Tree)
----------	--

Hình 3.6	Sơ đồ hoạt động của hệ thống
----------	------------------------------

Hình 4.1	Cài đặt thư viện Scikit-learn
----------	-------------------------------

Hình 4.2	Kết quả đọc dữ liệu từ file bugs.csv
----------	--------------------------------------

Hình 4.3	Quá trình huấn luyện mô hình Decision Tree
----------	--

Hình 4.4	Kết quả đánh giá độ chính xác của mô hình
----------	---

Hình 4.5	Kết quả dự đoán mức độ nghiêm trọng của lỗi
----------	---

LỜI CAM ĐOAN

Em xin cam đoan rằng tiểu luận “Nền tảng AI/ML cần biết cho kiểm thử phần mềm hiện đại” là kết quả nghiên cứu, tìm hiểu và thực hiện của cá nhân em trong quá trình học tập môn Kiểm thử phần mềm.

Các nội dung, số liệu, hình ảnh và kết quả thực nghiệm được trình bày trong báo cáo là trung thực, được tham khảo từ các nguồn tài liệu đáng tin cậy và đã được trích dẫn đầy đủ trong phần tài liệu tham khảo. Những nội dung thực nghiệm, xây dựng mô hình Machine Learning và đánh giá kết quả đều do cá nhân em thực hiện.

Em xin chịu hoàn toàn trách nhiệm về tính trung thực và chính xác của nội dung báo cáo này.

Tên sinh viên

LỜI NÓI ĐẦU

Trong thời đại chuyển đổi số hiện nay, phần mềm đóng vai trò quan trọng trong hầu hết các lĩnh vực của đời sống và sản xuất. Chất lượng phần mềm là một trong những yếu tố quyết định đến sự thành công của một hệ thống thông tin. Vì vậy, hoạt động kiểm thử phần mềm ngày càng được chú trọng nhằm phát hiện lỗi, đảm bảo tính ổn định và nâng cao chất lượng sản phẩm trước khi đưa vào sử dụng.

Môn học Kiểm thử phần mềm cung cấp cho sinh viên những kiến thức nền tảng về quy trình kiểm thử, các kỹ thuật thiết kế kiểm thử, quản lý lỗi và các phương pháp đảm bảo chất lượng phần mềm. Bên cạnh các phương pháp kiểm thử truyền thống, sự phát triển mạnh mẽ của Trí tuệ nhân tạo (Artificial Intelligence – AI) và Học máy (Machine Learning – ML) đã mở ra nhiều hướng tiếp cận mới trong lĩnh vực kiểm thử phần mềm.

Nhận thấy tiềm năng to lớn của AI/ML trong lĩnh vực đảm bảo chất lượng phần mềm, em đã lựa chọn đề tài: *“Ứng dụng AI/ML trong kiểm thử phần mềm hiện đại và thực nghiệm phân loại mức độ nghiêm trọng của lỗi phần mềm bằng Machine Learning”* để nghiên cứu và thực hiện. Đề tài tập trung tìm hiểu các khái niệm cơ bản về AI, Machine Learning, các phương pháp học có giám sát và học không giám sát, đồng thời xây dựng một mô hình học máy đơn giản nhằm hỗ trợ phân loại mức độ nghiêm trọng của lỗi phần mềm.

Thông qua quá trình thực hiện đề tài, em có cơ hội vận dụng những kiến thức đã học trong môn Kiểm thử phần mềm, đồng thời tiếp cận với các công nghệ AI đang được ứng dụng rộng rãi trong thực tế. Mặc dù đã cố gắng hoàn thành đề tài một cách tốt nhất, nhưng do kiến thức và thời gian thực hiện còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp của giảng viên để báo cáo được hoàn thiện hơn. Em xin chân thành cảm ơn!

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Giới thiệu môn học

Kiểm thử phần mềm là một trong những hoạt động quan trọng trong quy trình phát triển phần mềm. Mục đích chính của kiểm thử phần mềm là phát hiện lỗi, đánh giá mức độ đáp ứng yêu cầu và đảm bảo chất lượng của sản phẩm trước khi đưa vào sử dụng thực tế. Một phần mềm dù được thiết kế tốt nhưng nếu không được kiểm thử đầy đủ thì vẫn có thể tồn tại nhiều lỗi ảnh hưởng đến người dùng, dữ liệu và hoạt động của hệ thống.

Môn học Kiểm thử phần mềm cung cấp cho sinh viên các kiến thức nền tảng về quy trình kiểm thử, các mức kiểm thử, kỹ thuật thiết kế test case, quản lý lỗi và đánh giá chất lượng phần mềm. Thông qua môn học này, sinh viên hiểu được vai trò của kiểm thử trong việc giảm thiểu rủi ro, nâng cao độ tin cậy và đảm bảo phần mềm hoạt động đúng theo yêu cầu đặt ra.

Trong thực tế, hoạt động kiểm thử phần mềm không chỉ dừng lại ở việc chạy thử chương trình để tìm lỗi. Kiểm thử còn bao gồm việc phân tích yêu cầu, xây dựng kịch bản kiểm thử, ghi nhận lỗi, đánh giá mức độ nghiêm trọng của lỗi và ưu tiên xử lý lỗi. Đối với các hệ thống phần mềm lớn, số lượng lỗi phát sinh có thể rất nhiều, khiến quá trình quản lý và phân loại lỗi trở nên khó khăn nếu chỉ thực hiện thủ công.

Hiện nay, cùng với sự phát triển của công nghệ, Trí tuệ nhân tạo và Học máy đang được ứng dụng ngày càng nhiều trong lĩnh vực kiểm thử phần mềm. Các công nghệ này có khả năng hỗ trợ tự động hóa một số công việc của QA như phân tích lỗi, dự đoán rủi ro, phát hiện bất thường và phân loại mức độ nghiêm trọng của lỗi. Đây là một hướng tiếp cận mới giúp nâng cao hiệu quả kiểm thử phần mềm trong bối cảnh hiện đại.

Vì vậy, việc tìm hiểu nền tảng AI/ML trong kiểm thử phần mềm là cần thiết đối với sinh viên ngành Công nghệ thông tin. Nội dung này giúp sinh viên không chỉ nắm được kiến thức lý thuyết về kiểm thử phần mềm mà còn biết

cách vận dụng các kỹ thuật học máy đơn giản vào một bài toán cụ thể trong hoạt động QA.

1.2. Lý do chọn đề tài

Trong những năm gần đây, Trí tuệ nhân tạo và Học máy đã có sự phát triển mạnh mẽ và được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau. Trong lĩnh vực công nghệ phần mềm, AI/ML không chỉ được sử dụng để xây dựng các hệ thống thông minh mà còn được áp dụng vào quá trình phát triển, kiểm thử và bảo trì phần mềm.

Đối với kiểm thử phần mềm, đội ngũ QA thường phải xử lý một số lượng lớn lỗi phát sinh trong quá trình phát triển hệ thống. Mỗi lỗi có mức độ ảnh hưởng khác nhau. Có lỗi chỉ ảnh hưởng nhỏ đến giao diện hoặc trải nghiệm người dùng, nhưng cũng có lỗi làm hệ thống bị sập, gây mất dữ liệu hoặc làm gián đoạn hoạt động của phần mềm. Vì vậy, việc xác định mức độ nghiêm trọng của lỗi là công việc quan trọng để QA và lập trình viên ưu tiên xử lý các lỗi nguy hiểm trước.

Tuy nhiên, nếu phân loại lỗi hoàn toàn bằng phương pháp thủ công thì có thể mất nhiều thời gian, đặc biệt khi số lượng lỗi lớn. Ngoài ra, việc đánh giá mức độ nghiêm trọng của lỗi còn phụ thuộc vào kinh nghiệm của người kiểm thử, dẫn đến khả năng đánh giá chưa thống nhất giữa các thành viên trong nhóm.

Từ thực tế đó, đề tài “Nền tảng AI/ML cần biết cho kiểm thử phần mềm hiện đại” được lựa chọn nhằm tìm hiểu các kiến thức cơ bản về AI/ML và khả năng ứng dụng của chúng trong hoạt động kiểm thử phần mềm. Bên cạnh phần lý thuyết, đề tài còn xây dựng một chương trình mô phỏng bằng Python sử dụng thuật toán Decision Tree để phân loại mức độ nghiêm trọng của lỗi phần mềm. Cụ thể, bài thực nghiệm sử dụng bộ dữ liệu mẫu trong file bugs.csv. Bộ dữ liệu gồm các thuộc tính Bug_Count, Crash và Severity. Trong đó, Bug_Count thể hiện số lượng lỗi được phát hiện, Crash thể hiện trạng thái hệ thống có bị sập hay không, còn Severity là mức độ nghiêm trọng của lỗi gồm Minor, Major và

Critical. Từ dữ liệu này, mô hình Machine Learning được huấn luyện để dự đoán mức độ nghiêm trọng của một lỗi mới.

Việc lựa chọn đề tài này giúp người học hiểu rõ hơn mối liên hệ giữa kiểm thử phần mềm và AI/ML, đồng thời biết cách vận dụng một thuật toán học máy cơ bản vào một tình huống QA cụ thể.

1.3. Mục tiêu nghiên cứu

Đề tài được thực hiện nhằm đạt được các mục tiêu chính sau:

Thứ nhất, tìm hiểu các khái niệm cơ bản về Trí tuệ nhân tạo và Học máy. Đây là nền tảng quan trọng để hiểu cách máy tính có thể học từ dữ liệu và đưa ra dự đoán dựa trên dữ liệu đã được huấn luyện.

Thứ hai, tìm hiểu các phương pháp học máy phổ biến gồm học có giám sát, học không giám sát và học tăng cường. Trong đó, học có giám sát là phương pháp được sử dụng trực tiếp trong phần thực nghiệm của đề tài vì bộ dữ liệu lỗi phần mềm đã có sẵn nhãn Severity.

Thứ ba, phân tích khả năng ứng dụng AI/ML trong kiểm thử phần mềm. Đề tài tập trung làm rõ AI/ML có thể hỗ trợ QA trong các hoạt động như phân loại lỗi phần mềm, dự đoán lỗi, phân tích log, hỗ trợ sinh test case và nâng cao hiệu quả đảm bảo chất lượng phần mềm.

Thứ tư, xây dựng một mô hình Machine Learning đơn giản để phân loại mức độ nghiêm trọng của lỗi phần mềm. Mô hình sử dụng thuật toán Decision Tree, dữ liệu đầu vào gồm Bug_Count và Crash, kết quả đầu ra là Severity.

Thứ năm, thực nghiệm mô hình bằng Python và thư viện Scikit-learn. Sau khi huấn luyện mô hình, tiến hành đánh giá bằng chỉ số Accuracy và thử nghiệm dự đoán một lỗi mới có Bug_Count = 8 và Crash = 1. Kết quả mô hình dự đoán lỗi thuộc nhóm Critical, phù hợp với đặc điểm dữ liệu đầu vào.

Thông qua các mục tiêu trên, đề tài hướng đến việc chứng minh rằng Machine Learning có thể được ứng dụng để hỗ trợ quá trình kiểm thử phần mềm, đặc biệt trong việc phân loại và ưu tiên xử lý lỗi.

1.4. Phạm vi nghiên cứu

Trong phạm vi của tiểu luận, đề tài tập trung nghiên cứu các kiến thức nền tảng về AI/ML và ứng dụng của chúng trong kiểm thử phần mềm hiện đại. Nội dung không đi sâu vào các mô hình phức tạp như Deep Learning, Computer Vision hoặc các hệ thống AI thương mại quy mô lớn.

Phần lý thuyết của đề tài tập trung vào các nội dung chính gồm Trí tuệ nhân tạo, Học máy, học có giám sát, học không giám sát, học tăng cường và thuật toán Decision Tree. Đây là những kiến thức phù hợp với yêu cầu tìm hiểu nền tảng AI/ML trong kiểm thử phần mềm.

Phần thực nghiệm của đề tài được giới hạn trong bài toán phân loại mức độ nghiêm trọng của lỗi phần mềm. Bộ dữ liệu sử dụng là dữ liệu mẫu được lưu trong file bugs.csv, gồm 16 mẫu dữ liệu. Mỗi mẫu dữ liệu có ba thuộc tính chính:

Bug_Count: số lượng lỗi được phát hiện.

Crash: trạng thái hệ thống có bị sập hay không.

Severity: mức độ nghiêm trọng của lỗi.

Mô hình được xây dựng bằng ngôn ngữ lập trình Python, sử dụng thư viện Pandas để đọc dữ liệu và thư viện Scikit-learn để huấn luyện thuật toán Decision Tree. Kết quả mô hình được đánh giá thông qua độ chính xác Accuracy và thử nghiệm dự đoán lỗi mới.

Do dữ liệu sử dụng trong đề tài là dữ liệu mẫu, kết quả thực nghiệm chủ yếu nhằm minh họa cách áp dụng Machine Learning vào kiểm thử phần mềm. Đề tài chưa áp dụng trên dữ liệu lỗi thực tế từ các dự án phần mềm lớn.

1.5. Phương pháp nghiên cứu

Để thực hiện đề tài, các phương pháp nghiên cứu được sử dụng gồm nghiên cứu tài liệu, xây dựng dữ liệu thực nghiệm, xây dựng mô hình Machine Learning và đánh giá kết quả.

1.5.1. Nghiên cứu tài liệu

- Trước tiên, đề tài tiến hành tìm hiểu các tài liệu liên quan đến kiểm thử phần mềm, Trí tuệ nhân tạo, Học máy và các ứng dụng AI/ML trong lĩnh vực QA. Nội dung nghiên cứu tập trung vào các khái niệm cơ bản, các loại học máy phổ biến và vai trò của AI/ML trong việc hỗ trợ hoạt động kiểm thử phần mềm.
- Việc nghiên cứu tài liệu giúp làm rõ cơ sở lý thuyết của đề tài, từ đó lựa chọn được phương pháp phù hợp để áp dụng vào bài toán thực nghiệm. Trong đề tài này, học có giám sát được lựa chọn vì bộ dữ liệu có sẵn nhãn kết quả là Severity.

1.5.2. Thu thập và xây dựng dữ liệu

Do đề tài mang tính mô phỏng, bộ dữ liệu được xây dựng dưới dạng dữ liệu mẫu. Dữ liệu được lưu trong file bugs.csv và gồm các thuộc tính Bug_Count, Crash và Severity.

- Dữ liệu này được sử dụng làm cơ sở để huấn luyện mô hình học máy. Trong đó, Bug_Count và Crash là dữ liệu đầu vào, còn Severity là nhãn dữ liệu mà mô hình cần học và dự đoán.
- Việc xây dựng dữ liệu mẫu giúp mô phỏng một tình huống thực tế trong kiểm thử phần mềm, đó là phân loại mức độ nghiêm trọng của lỗi dựa trên thông tin về số lượng lỗi và trạng thái hệ thống bị sập.

1.5.3. Xây dựng mô hình Machine Learning

Sau khi chuẩn bị dữ liệu, đề tài sử dụng Python để xây dựng mô hình Machine Learning. Thư viện Pandas được sử dụng để đọc và xử lý dữ liệu từ file bugs.csv. Thư viện Scikit-learn được sử dụng để chia dữ liệu, xây dựng mô hình Decision Tree và đánh giá kết quả.

- Thuật toán Decision Tree được lựa chọn vì đây là thuật toán học có giám sát, phù hợp với bài toán phân loại. Ngoài ra, thuật toán này có cách hoạt động dễ hiểu, dễ triển khai và phù hợp với bộ dữ liệu nhỏ trong đề tài.

- Mô hình sử dụng hai thuộc tính đầu vào là Bug_Count và Crash để dự đoán nhãn đầu ra là Severity. Sau khi được huấn luyện, mô hình có thể dự đoán một lỗi mới thuộc nhóm Minor, Major hoặc Critical.

1.5.4. Đánh giá kết quả

Sau khi huấn luyện mô hình, đề tài tiến hành đánh giá kết quả thông qua chỉ số Accuracy. Đây là chỉ số cho biết tỷ lệ dự đoán đúng của mô hình trên tập dữ liệu kiểm tra.

- Trong phần thực nghiệm, mô hình đạt độ chính xác 75%. Kết quả này cho thấy mô hình đã học được một số quy luật cơ bản từ dữ liệu, tuy nhiên do bộ dữ liệu còn nhỏ nên độ chính xác chưa đạt mức tối ưu.
- Ngoài ra, mô hình được thử nghiệm với dữ liệu mới gồm Bug_Count = 8 và Crash = 1. Kết quả dự đoán là Critical - lỗi nghiêm trọng. Kết quả này phù hợp với thực tế vì lỗi có số lượng cao và làm hệ thống bị sập thường là lỗi có mức độ nghiêm trọng cao.

Tóm tắt chương 1

Chương 1 đã trình bày tổng quan về đề tài “Nền tảng AI/ML cần biết cho kiểm thử phần mềm hiện đại”. Nội dung chương đã giới thiệu vai trò của môn học Kiểm thử phần mềm, lý do lựa chọn đề tài, mục tiêu nghiên cứu, phạm vi nghiên cứu và phương pháp thực hiện.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Trí tuệ nhân tạo trong kiểm thử phần mềm

Trí tuệ nhân tạo, hay Artificial Intelligence (AI), là lĩnh vực của khoa học máy tính nghiên cứu và xây dựng các hệ thống có khả năng mô phỏng một số hoạt động trí tuệ của con người như học tập, suy luận, nhận dạng dữ liệu, phân tích thông tin và đưa ra quyết định. Thay vì chỉ thực hiện các lệnh cố định do con người lập trình sẵn, hệ thống AI có thể xử lý dữ liệu, nhận ra quy luật và hỗ trợ con người trong quá trình ra quyết định.

Trong lĩnh vực kiểm thử phần mềm, AI được ứng dụng nhằm hỗ trợ các hoạt động như phân tích lỗi, sinh test case, phát hiện bất thường, dự đoán khu vực có nguy cơ lỗi cao và đánh giá mức độ nghiêm trọng của lỗi phần mềm. Điều này giúp quá trình kiểm thử trở nên nhanh hơn, giảm công việc thủ công và nâng cao hiệu quả đảm bảo chất lượng phần mềm.

Đối với đề tài này, AI được xem là nền tảng lý thuyết để xây dựng một mô hình học máy có khả năng hỗ trợ QA phân loại mức độ nghiêm trọng của lỗi phần mềm. Cụ thể, mô hình sẽ dựa trên dữ liệu lỗi đã có để dự đoán lỗi mới thuộc nhóm Minor, Major hay Critical.

2.1.1. Đặc điểm của AI

AI có một số đặc điểm cơ bản sau:

- Có khả năng học từ dữ liệu.
- Có khả năng nhận diện mẫu và quy luật trong dữ liệu.
- Có thể hỗ trợ đưa ra dự đoán hoặc quyết định.
- Có thể tự động hóa một số công việc lặp lại.
- Có khả năng xử lý lượng dữ liệu lớn nhanh hơn con người.

Trong kiểm thử phần mềm, những đặc điểm này giúp AI trở thành công cụ hỗ trợ hiệu quả cho đội ngũ QA, đặc biệt trong các công việc liên quan đến phân tích dữ liệu lỗi và đánh giá rủi ro.

2.1.2. Vai trò của AI trong kiểm thử phần mềm

Đảm bảo chất lượng phần mềm không chỉ là việc tìm lỗi mà còn bao gồm việc kiểm soát rủi ro, đánh giá độ ổn định và nâng cao độ tin cậy của hệ thống. AI có thể hỗ trợ quá trình này thông qua việc phân tích dữ liệu kiểm thử và đưa ra những gợi ý có giá trị.

Ví dụ, nếu một hệ thống có nhiều lỗi nghiêm trọng liên quan đến chức năng thanh toán, AI có thể hỗ trợ xác định đây là khu vực có rủi ro cao để QA tập trung kiểm thử kỹ hơn. Ngoài ra, AI cũng có thể hỗ trợ tự động phân loại lỗi theo mức độ nghiêm trọng, giúp nhóm phát triển ưu tiên xử lý các lỗi quan trọng trước.

Trong bài tiểu luận này, vai trò của AI được thể hiện thông qua việc ứng dụng Machine Learning để hỗ trợ phân loại mức độ nghiêm trọng của lỗi phần mềm. Đây là một ví dụ đơn giản nhưng phù hợp với thực tế hoạt động QA

2.2. Học máy (Machine Learning - ML)

Machine Learning (ML), hay học máy, là một nhánh của trí tuệ nhân tạo. Machine Learning cho phép máy tính học từ dữ liệu để tìm ra quy luật và đưa ra dự đoán mà không cần lập trình cụ thể cho từng trường hợp.

Khác với cách lập trình truyền thống, trong đó lập trình viên phải viết sẵn các quy tắc xử lý, Machine Learning sử dụng dữ liệu để mô hình tự học. Sau khi được huấn luyện, mô hình có thể áp dụng những quy luật đã học để xử lý dữ liệu mới.

Trong đề tài này, Machine Learning được sử dụng để giải quyết bài toán phân loại mức độ nghiêm trọng của lỗi phần mềm. Dữ liệu đầu vào gồm Bug_Count và Crash, dữ liệu đầu ra là Severity. Mô hình sẽ học từ dữ liệu có sẵn trong file bugs.csv để dự đoán mức độ lỗi mới.

2.2.1. Quy trình xây dựng mô hình Machine Learning

Một mô hình Machine Learning thường được xây dựng qua các bước cơ bản sau:

Bước 1: Thu thập dữ liệu

Dữ liệu là yếu tố quan trọng nhất trong học máy. Trong đề tài này, dữ liệu được xây dựng dưới dạng file bugs.csv gồm các thông tin về lỗi phần mềm như số lượng lỗi, trạng thái hệ thống bị sập và mức độ nghiêm trọng của lỗi.

Bước 2: Tiền xử lý dữ liệu

Dữ liệu cần được kiểm tra và chuyển về dạng phù hợp để mô hình có thể học được. Trong bài thực nghiệm, dữ liệu được đọc bằng thư viện Pandas. Các cột Bug_Count và Crash được sử dụng làm dữ liệu đầu vào, còn Severity là nhãn cần dự đoán.

Bước 3: Chia dữ liệu

Dữ liệu được chia thành tập huấn luyện và tập kiểm tra. Tập huấn luyện dùng để mô hình học quy luật, còn tập kiểm tra dùng để đánh giá khả năng dự đoán của mô hình trên dữ liệu chưa từng học.

Bước 4: Huấn luyện mô hình

Mô hình được huấn luyện bằng thuật toán Decision Tree. Quá trình huấn luyện giúp mô hình học mối quan hệ giữa Bug_Count, Crash và Severity.

Bước 5: Đánh giá mô hình

Sau khi huấn luyện, mô hình được đánh giá bằng chỉ số Accuracy. Trong bài thực nghiệm, mô hình đạt độ chính xác 75%.

Bước 6: Dự đoán dữ liệu mới

Sau khi mô hình được huấn luyện, có thể đưa dữ liệu lỗi mới vào để dự đoán mức độ nghiêm trọng. Ví dụ, với Bug_Count = 8 và Crash = 1, mô hình dự đoán kết quả là Critical

2.2.2. Liên hệ Machine Learning với kiểm thử phần mềm

Machine Learning có thể hỗ trợ kiểm thử phần mềm bằng cách xử lý và phân tích dữ liệu kiểm thử.

Trong thực tế, các dự án phần mềm thường có rất nhiều bug report, log hệ thống và kết quả test case. Nếu phân tích thủ công, QA sẽ mất nhiều thời gian.

Khi áp dụng Machine Learning, hệ thống có thể học từ dữ liệu lỗi trong quá khứ để hỗ trợ các công việc như phân loại lỗi, dự đoán module có nguy cơ lỗi cao hoặc phát hiện bất thường trong log. Điều này góp phần nâng cao chất lượng phần mềm vì lỗi được phát hiện và ưu tiên xử lý hiệu quả hơn.

2.3. Học có giám sát (Supervised Learning)

Học có giám sát, hay Supervised Learning, là phương pháp học máy sử dụng dữ liệu đã được gán nhãn. Mỗi mẫu dữ liệu đầu vào đều có sẵn kết quả đúng tương ứng. Mô hình sẽ học mối quan hệ giữa dữ liệu đầu vào và nhãn đầu ra để dự đoán kết quả cho dữ liệu mới.

Trong đề tài này, học có giám sát là phương pháp được sử dụng trực tiếp trong phần thực nghiệm. Bộ dữ liệu bugs.csv đã có sẵn nhãn Severity cho từng mẫu lỗi, vì vậy mô hình có thể học từ các ví dụ này để phân loại lỗi mới.

2.3.1. Nguyên lý hoạt động

Dữ liệu trong học có giám sát gồm hai phần:

- Input: dữ liệu đầu vào.
- Output: kết quả đầu ra hoặc nhãn dữ liệu.

Trong bài toán của đề tài:

- Input gồm Bug_Count và Crash.
- Output là Severity.

Ví dụ:

Bug_Count	Crash	Severity
1	0	Minor
5	0	Major
8	1	Critical

Trong bảng trên, Bug_Count và Crash là dữ liệu đầu vào, còn Severity là nhãn dữ liệu. Mô hình sẽ học từ các mẫu này để rút ra quy luật.

Có thể hiểu đơn giản như sau:

- Nếu Bug_Count nhỏ và Crash = 0 thì lỗi có thể là Minor.
- Nếu Bug_Count lớn hơn nhưng không làm hệ thống sập thì lỗi có thể là Major.
- Nếu Crash = 1 thì lỗi có khả năng là Critical.

Sau khi học các quy luật này, mô hình có thể dự đoán mức độ nghiêm trọng cho lỗi mới.

2.3.2. Ứng dụng trong đề tài

Trong đề tài này, học có giám sát được sử dụng để xây dựng mô hình phân loại mức độ nghiêm trọng của lỗi phần mềm. Bộ dữ liệu mẫu được lưu trong file bugs.csv, trong đó mỗi dòng dữ liệu đều có sẵn nhãn Severity gồm Minor, Major hoặc Critical.

Mô hình Machine Learning sử dụng thuật toán Decision Tree để học mối quan hệ giữa hai thuộc tính đầu vào là Bug_Count và Crash với nhãn đầu ra là Severity. Sau khi được huấn luyện, mô hình có thể dự đoán mức độ nghiêm trọng của lỗi mới

- Minor (Lỗi nhẹ).
- Major (Lỗi trung bình).
- Critical (Lỗi nghiêm trọng).

đối với các lỗi mới phát sinh.

2.3.3. Ưu điểm

Học có giám sát có một số ưu điểm:

Phù hợp với bài toán phân loại và dự đoán.

Dễ đánh giá kết quả vì dữ liệu đã có sẵn nhãn đúng.

Có thể đạt độ chính xác cao nếu dữ liệu huấn luyện đầy đủ.

Phù hợp với bài toán phân loại mức độ nghiêm trọng của lỗi phần mềm.

2.3.4. Nhược điểm

Học có giám sát cũng có một số hạn chế:

Cần dữ liệu đã được gắn nhãn trước.

Nếu dữ liệu ít thì mô hình có thể học chưa tốt.

Chất lượng mô hình phụ thuộc nhiều vào chất lượng dữ liệu.

Nếu dữ liệu không phản ánh đúng thực tế, kết quả dự đoán có thể sai lệch.

Trong bài thực nghiệm, bộ dữ liệu chỉ gồm 16 mẫu nên mô hình mới chỉ mang tính mô phỏng. Vì vậy, kết quả Accuracy 75% cho thấy mô hình đã học được quy luật cơ bản, nhưng vẫn cần nhiều dữ liệu thực tế hơn để nâng cao độ chính xác.

2.4. Học không giám sát (Unsupervised Learning)

Học không giám sát, hay Unsupervised Learning, là phương pháp học máy sử dụng dữ liệu chưa được gắn nhãn. Khác với học có giám sát, dữ liệu đầu vào không có sẵn kết quả đúng. Mô hình sẽ tự tìm ra các mối quan hệ hoặc tự chia dữ liệu thành các nhóm có đặc điểm tương đồng.

Trong lĩnh vực kiểm thử phần mềm, học không giám sát có thể được sử dụng để gom nhóm lỗi phần mềm, phân tích log hoặc phát hiện các hành vi bất thường của hệ thống.

2.4.1. Nguyên lý hoạt động

Trong học không giám sát, mô hình chỉ nhận dữ liệu đầu vào và không biết trước kết quả đầu ra.

Ví dụ:

Bug_Count
1
2
3
8
9
10

Với dữ liệu trên, mô hình có thể tự chia thành hai nhóm:

Nhóm 1: Các lỗi có Bug_Count thấp như 1, 2, 3.

Nhóm 2: Các lỗi có Bug_Count cao như 8, 9, 10.

Như vậy, mô hình không cần biết trước lỗi thuộc loại Minor, Major hay Critical mà vẫn có thể tự phân nhóm dữ liệu dựa trên mức độ tương đồng.

2.4.2. Ứng dụng trong QA

Trong hoạt động QA, học không giám sát có thể được áp dụng vào các công việc như:

- Gom nhóm các lỗi phần mềm có đặc điểm giống nhau.
- Gom nhóm các test case tương tự để dễ quản lý.
- Phân tích log hệ thống nhằm tìm ra các mẫu bất thường.
- Phát hiện hành vi bất thường của hệ thống trong quá trình kiểm thử.
- Tuy nhiên, trong đề tài này, học không giám sát chỉ được trình bày ở phần lý thuyết. Phần thực nghiệm của đề tài sử dụng học có giám sát với thuật toán Decision Tree để phân loại mức độ nghiêm trọng của lỗi phần mềm.

2.4.3. Ưu điểm

Học không giám sát có các ưu điểm:

- Không cần dữ liệu đã gán nhãn.

- Có thể khám phá các quy luật tiềm ẩn trong dữ liệu.
- Hữu ích khi dữ liệu lớn nhưng chưa được phân loại.
- Có thể hỗ trợ QA phát hiện các nhóm lỗi hoặc hành vi bất thường.

2.4.4. Nhược điểm

Hạn chế của học không giám sát là:

- Khó đánh giá độ chính xác vì không có đáp án đúng để so sánh.
- Kết quả phân nhóm có thể khó giải thích.
- Cần người dùng phân tích lại ý nghĩa của từng nhóm.
- Không phù hợp nếu mục tiêu là dự đoán một nhãn cụ thể đã biết.

Trong đề tài này, học không giám sát chỉ được trình bày ở phần lý thuyết để làm rõ các loại AI/ML cần biết trong kiểm thử phần mềm hiện đại. Phần thực nghiệm không sử dụng học không giám sát mà sử dụng học có giám sát.

2.5. Học tăng cường (Reinforcement Learning)

Học tăng cường, hay Reinforcement Learning, là phương pháp học máy trong đó mô hình học thông qua quá trình thử nghiệm và nhận phản hồi từ môi trường. Nếu hành động mang lại kết quả tốt, mô hình sẽ nhận được phần thưởng. Nếu hành động không tốt, mô hình sẽ nhận phản hồi tiêu cực.

Phương pháp này thường được sử dụng trong các bài toán cần tối ưu hành động, ví dụ như robot tự học di chuyển, trò chơi, điều khiển hệ thống hoặc tối ưu chiến lược.

2.5.1. Liên hệ với kiểm thử phần mềm

Trong kiểm thử phần mềm, học tăng cường có thể được dùng để tối ưu thứ tự chạy test case. Ví dụ, nếu một số test case thường phát hiện lỗi nhiều hơn, hệ thống có thể học để ưu tiên chạy các test case đó trước. Nhờ vậy, QA có thể phát hiện lỗi quan trọng nhanh hơn.

Học tăng cường cũng có thể hỗ trợ tìm đường đi kiểm thử hiệu quả trong các hệ thống có nhiều chức năng và nhiều luồng xử lý.

2.5.2. Ưu điểm và giới hạn

Ưu điểm của học tăng cường là có khả năng tự tối ưu chiến lược sau nhiều lần thử nghiệm. Tuy nhiên, phương pháp này thường phức tạp, cần nhiều dữ liệu và thời gian huấn luyện dài.

Trong đề tài này, học tăng cường chỉ được giới thiệu ở mức lý thuyết, không được sử dụng trong phần thực nghiệm.

2.6. Thuật toán Decision Tree

Decision Tree, hay cây quyết định, là thuật toán Machine Learning thuộc nhóm học có giám sát. Thuật toán này được sử dụng phổ biến trong các bài toán phân loại vì có cách hoạt động dễ hiểu và dễ giải thích.

- Trong đề tài này, Decision Tree là thuật toán được sử dụng trực tiếp để xây dựng mô hình phân loại mức độ nghiêm trọng của lỗi phần mềm.
- Trong đề tài này, thuật toán Decision Tree được sử dụng để phân loại mức độ nghiêm trọng của lỗi phần mềm.

2.6.1. Cơ chế hoạt động của Decision Tree

Decision Tree hoạt động bằng cách tạo ra một cây gồm các nút quyết định. Mỗi nút là một điều kiện kiểm tra dữ liệu. Dựa vào kết quả của điều kiện, dữ liệu sẽ đi theo các nhánh khác nhau cho đến khi đến nút lá, nơi đưa ra kết quả phân loại cuối cùng.

Trong bài toán của đề tài, mô hình sử dụng hai thuộc tính đầu vào:

- Bug_Count.
- Crash.

Kết quả đầu ra là:

- Severity.

Có thể mô tả đơn giản như sau:

Nếu Crash = 1 → Lỗi có khả năng thuộc nhóm Critical.

Nếu Crash = 0 và Bug_Count nhỏ → Lỗi có khả năng thuộc nhóm Minor.

Nếu Crash = 0 và Bug_Count lớn hơn → Lỗi có khả năng thuộc nhóm Major.

Cách hoạt động này phù hợp với bài toán vì QA cũng thường đánh giá lỗi dựa trên mức độ ảnh hưởng đến hệ thống. Nếu lỗi làm hệ thống bị sập thì lỗi đó thường nghiêm trọng hơn lỗi chỉ ảnh hưởng nhỏ.

2.6.2. Lý do lựa chọn Decision Tree trong đề tài

Thuật toán Decision Tree được lựa chọn vì:

- Dễ triển khai bằng Python và thư viện Scikit-learn.
- Phù hợp với bài toán phân loại.
- Phù hợp với bộ dữ liệu nhỏ trong file bugs.csv.
- Kết quả dễ giải thích khi trình bày trong báo cáo.
- Có thể minh họa bằng sơ đồ cây quyết định.

Trong bài thực nghiệm, mô hình Decision Tree được huấn luyện từ dữ liệu đã có nhãn Severity. Sau đó, mô hình được dùng để dự đoán lỗi mới dựa trên Bug_Count và Crash.

2.6.3. Ưu điểm của Decision Tree

Decision Tree có các ưu điểm:

- Cấu trúc đơn giản, dễ hiểu.
- Dễ trực quan hóa bằng sơ đồ cây.
- Phù hợp với bài toán phân loại.
- Không yêu cầu xử lý dữ liệu quá phức tạp.
- Dễ giải thích lý do mô hình đưa ra dự đoán.

2.6.4. Nhược điểm của Decision Tree

Bên cạnh ưu điểm, Decision Tree cũng có một số hạn chế:

- Nếu dữ liệu ít, mô hình có thể dự đoán chưa ổn định.
- Nếu dữ liệu quá phức tạp, cây có thể bị quá khớp.
- Kết quả phụ thuộc vào chất lượng dữ liệu huấn luyện.
- Với bài toán thực tế lớn, cần so sánh thêm với các thuật toán khác như Random Forest hoặc Logistic Regression.

Trong đề tài này, do dữ liệu chỉ gồm 16 mẫu, Decision Tree được sử dụng chủ yếu để mô phỏng cách Machine Learning hỗ trợ phân loại lỗi phần mềm.

2.7. Ứng dụng AI/ML trong kiểm thử phần mềm

AI/ML có thể được ứng dụng trong nhiều hoạt động kiểm thử phần mềm nhằm nâng cao chất lượng và hiệu quả kiểm thử.

2.7.1. Tự động sinh test case

AI có thể hỗ trợ phân tích yêu cầu phần mềm và đề xuất các test case phù hợp. Điều này giúp QA tiết kiệm thời gian trong quá trình thiết kế kịch bản kiểm thử.

Ví dụ, với chức năng đăng nhập, AI có thể gợi ý các trường hợp kiểm thử như nhập đúng tài khoản, nhập sai mật khẩu, bỏ trống dữ liệu hoặc nhập ký tự đặc biệt.

2.7.2. Phân loại lỗi phần mềm

Đây là ứng dụng liên quan trực tiếp đến đề tài. Machine Learning có thể học từ dữ liệu lỗi đã có để phân loại lỗi mới theo mức độ nghiêm trọng.

Trong bài thực nghiệm, mô hình sử dụng Bug_Count và Crash để dự đoán Severity. Việc này giúp QA nhanh chóng xác định lỗi nào cần xử lý trước.

2.7.3. Dự đoán lỗi

AI có thể phân tích dữ liệu từ các phiên bản phần mềm trước đó để dự đoán khu vực nào có nguy cơ phát sinh lỗi cao. Nhờ đó, QA có thể tập trung kiểm thử kỹ hơn vào các chức năng quan trọng.

Ví dụ, nếu module thanh toán thường xuyên phát sinh lỗi, AI có thể cảnh báo đây là khu vực cần kiểm thử nhiều hơn.

2.7.4. Phân tích log

Trong các hệ thống phần mềm, log thường có số lượng rất lớn. AI có thể hỗ trợ phát hiện các dòng log bất thường hoặc các lỗi lặp lại nhiều lần. Điều này giúp QA và lập trình viên tìm nguyên nhân lỗi nhanh hơn.

2.7.5. Kiểm thử giao diện

AI có thể hỗ trợ kiểm thử giao diện bằng cách so sánh giao diện giữa các phiên bản phần mềm. Nếu có sự thay đổi bất thường như sai màu, lệch nút hoặc mất nội dung, hệ thống có thể phát hiện và cảnh báo.

Tuy nhiên, trong đề tài này, phần thực nghiệm chỉ tập trung vào phân loại mức độ nghiêm trọng của lỗi phần mềm bằng Decision Tree, chưa đi sâu vào kiểm thử giao diện.

2.8. Liên hệ với chất lượng phần mềm

Các kiến thức AI/ML trình bày trong chương này có liên hệ trực tiếp đến hoạt động đảm bảo chất lượng phần mềm. Trong quá trình kiểm thử, QA cần phát hiện lỗi, đánh giá mức độ nghiêm trọng của lỗi và ưu tiên xử lý các lỗi quan trọng. AI/ML có thể hỗ trợ quá trình này bằng cách phân tích dữ liệu lỗi và đưa ra dự đoán nhanh hơn.

Việc ứng dụng Machine Learning vào phân loại lỗi giúp giảm thời gian đánh giá thủ công, hỗ trợ QA ra quyết định và góp phần nâng cao hiệu quả kiểm thử. Nếu mô hình được huấn luyện bằng dữ liệu thực tế đầy đủ hơn, hệ thống có thể hỗ trợ tốt hơn trong việc phân loại lỗi, dự đoán rủi ro và tối ưu quy trình kiểm thử phần mềm.

Trong đề tài này, mô hình Decision Tree tuy còn đơn giản nhưng đã thể hiện được khả năng ứng dụng AI/ML vào một ngữ cảnh QA cụ thể: phân loại mức độ nghiêm trọng của lỗi phần mềm.

Tóm tắt chương 2

Chương 2 đã trình bày cơ sở lý thuyết và tổng quan về AI/ML trong kiểm thử phần mềm. Nội dung chương bao gồm khái niệm Trí tuệ nhân tạo, Machine Learning, học có giám sát, học không giám sát, học tăng cường và thuật toán Decision Tree. Bên cạnh đó, chương cũng phân tích ưu điểm, nhược điểm, giới hạn của từng phương pháp và làm rõ mối liên hệ giữa AI/ML với hoạt động đảm bảo chất lượng phần mềm.

CHƯƠNG 3. PHÂN TÍCH BÀI TOÁN VÀ XÂY DỰNG MÔ HÌNH

3.1. Giới thiệu bài toán

Trong quá trình kiểm thử phần mềm, lỗi phát sinh là điều khó tránh khỏi. Mỗi lỗi có mức độ ảnh hưởng khác nhau đến hệ thống. Có lỗi chỉ ảnh hưởng nhỏ đến giao diện hoặc cách hiển thị thông tin, nhưng cũng có lỗi làm hệ thống bị sập, không thể tiếp tục hoạt động hoặc gây ảnh hưởng nghiêm trọng đến người dùng.

Vì vậy, việc xác định mức độ nghiêm trọng của lỗi là một nhiệm vụ quan trọng trong hoạt động kiểm thử phần mềm. Khi lỗi được phân loại đúng mức độ, đội ngũ QA và lập trình viên có thể ưu tiên xử lý các lỗi quan trọng trước, từ đó giúp nâng cao chất lượng phần mềm và giảm rủi ro khi triển khai hệ thống.

Trong thực tế, nếu số lượng lỗi ít thì QA có thể phân loại thủ công. Tuy nhiên, khi số lượng lỗi tăng lên, việc phân loại thủ công sẽ mất nhiều thời gian và có thể không thống nhất giữa các thành viên trong nhóm. Do đó, việc ứng dụng Machine Learning để hỗ trợ phân loại mức độ nghiêm trọng của lỗi là một hướng tiếp cận phù hợp.

Trong đề tài này, bài toán được xây dựng theo hướng sử dụng mô hình Machine Learning để phân loại lỗi phần mềm thành ba mức độ:

Minor: Lỗi nhẹ.

Major: Lỗi trung bình.

Critical: Lỗi nghiêm trọng.

Mô hình sẽ học từ dữ liệu lỗi đã có sẵn trong file bugs.csv, sau đó dự đoán mức độ nghiêm trọng của lỗi mới dựa trên các thông tin đầu vào.

3.2. Mô tả bài toán

Bài toán trong đề tài được xây dựng dưới dạng bài toán phân loại. Đây là dạng bài toán trong Machine Learning, trong đó mô hình sẽ dự đoán dữ liệu đầu vào thuộc nhóm nào trong các nhóm đã được định nghĩa trước.

Trong bài toán này, mô hình sử dụng hai thuộc tính đầu vào là Bug_Count và Crash để dự đoán thuộc tính đầu ra là Severity.

3.2.1. Dữ liệu đầu vào

Dữ liệu đầu vào của mô hình gồm:

Thuộc tính	Ý nghĩa
Bug_Count	Số lượng lỗi được phát hiện
Crash	Trạng thái lỗi có làm hệ thống bị sập hay không

Trong đó, Bug_Count cho biết số lượng lỗi được phát hiện trong quá trình kiểm thử. Giá trị này càng lớn thì hệ thống có thể đang tồn tại nhiều vấn đề hơn.

Thuộc tính Crash cho biết lỗi có làm hệ thống bị sập hay không. Nếu Crash = 0, hệ thống không bị sập. Nếu Crash = 1, hệ thống bị sập hoặc không thể tiếp tục hoạt động bình thường.

3.2.2. Dữ liệu đầu ra

Dữ liệu đầu ra của mô hình là Severity, thể hiện mức độ nghiêm trọng của lỗi phần mềm.

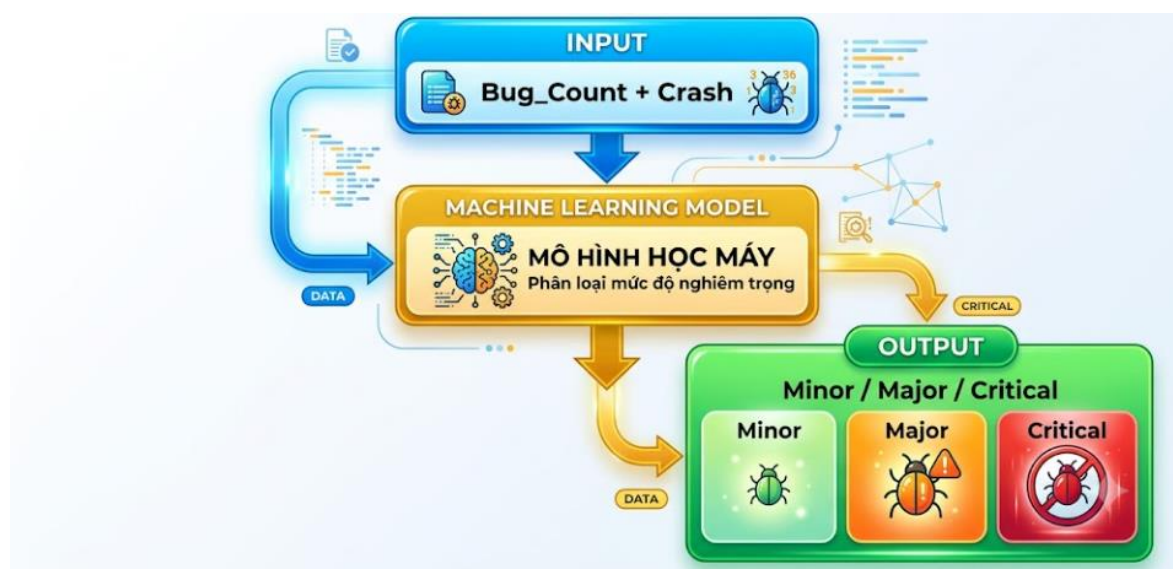
Severity	Ý nghĩa
Minor	Lỗi nhẹ
Major	Lỗi trung bình
Critical	Lỗi nghiêm trọng

Ví dụ dữ liệu:

Bug_Count	Crash	Severity
-----------	-------	----------

1	0	Minor
5	0	Major
8	1	Critical

Qua ví dụ trên có thể thấy, khi số lượng lỗi ít và không làm hệ thống bị sập thì lỗi có thể thuộc nhóm Minor. Khi số lượng lỗi nhiều hơn, lỗi có thể thuộc nhóm Major. Nếu lỗi làm hệ thống bị sập thì lỗi có khả năng thuộc nhóm Critical. Mục tiêu của mô hình là học mối quan hệ giữa Bug_Count, Crash và Severity, từ đó dự đoán mức độ nghiêm trọng của các lỗi mới phát sinh.



Hình 3.2 Mô hình bài toán phân loại lỗi phần mềm

3.3. Bộ dữ liệu sử dụng

Trong đề tài, bộ dữ liệu được xây dựng dưới dạng dữ liệu mẫu nhằm phục vụ cho quá trình thực nghiệm. Dữ liệu được lưu trong file bugs.csv.

Bộ dữ liệu gồm 16 mẫu lỗi phần mềm. Mỗi mẫu dữ liệu gồm ba thuộc tính là Bug_Count, Crash và Severity.

Thuộc tính	Ý nghĩa
Bug_Count	Số lượng lỗi phát hiện

Crash	Hệ thống có bị sập hay không
Severity	Mức độ nghiêm trọng của lỗi

Dữ liệu sử dụng trong file bugs.csv như sau.

Bug_Count	Crash	Severity
1	0	Minor
2	0	Minor
3	0	Minor
4	0	Major
5	0	Major
6	1	Critical
7	1	Critical
8	1	Critical
9	1	Critical
10	1	Critical
3	1	Major
4	1	Major
5	1	Critical
2	0	Minor
7	0	Major
8	1	Critical

Hình 3.2: Nội dung file bugs.csv

Trong bộ dữ liệu trên, Bug_Count và Crash là dữ liệu đầu vào, còn Severity là nhãn dữ liệu mà mô hình cần học và dự đoán. Vì dữ liệu đã có sẵn nhãn Severity, bài toán này thuộc nhóm học có giám sát

3.4. Phân tích dữ liệu

3.4.1. Thuộc tính Bug_Count

Bug_Count là thuộc tính thể hiện số lượng lỗi được phát hiện trong quá trình kiểm thử phần mềm. Thuộc tính này giúp mô hình nhận biết mức độ xuất hiện lỗi trong hệ thống.

Ví dụ:

Bug_Count
1
3
8

Nếu Bug_Count có giá trị nhỏ, lỗi thường có mức độ ảnh hưởng thấp hơn. Nếu Bug_Count có giá trị lớn, hệ thống có thể đang tồn tại nhiều lỗi hơn và lỗi có nguy cơ thuộc nhóm nghiêm trọng hơn.

Trong bài toán này, Bug_Count là một trong hai thuộc tính đầu vào quan trọng giúp mô hình phân loại mức độ nghiêm trọng của lỗi.

3.4.2. Thuộc tính Crash

Crash thể hiện trạng thái hệ thống có bị sập hay không.

Giá trị	Ý nghĩa
0	Không sập
1	Có sập

Thuộc tính Crash có ảnh hưởng lớn đến kết quả phân loại. Nếu Crash = 1, nghĩa là lỗi có thể làm hệ thống ngừng hoạt động. Trong trường hợp này, lỗi thường có khả năng thuộc nhóm Critical.

3.4.3. Thuộc tính Severity

Severity là nhãn dữ liệu, đồng thời là kết quả mà mô hình cần dự đoán. Thuộc tính này cho biết mức độ nghiêm trọng của lỗi phần mềm.

Severity	Ý nghĩa
Minor	Lỗi nhẹ
Major	Lỗi trung bình
Critical	Lỗi nghiêm trọng

Hình 3.3: Minh họa các mức độ lỗi phần mềm

Trong đề tài này, mô hình Machine Learning sẽ học từ các mẫu dữ liệu đã có sẵn nhãn Severity. Sau khi huấn luyện, mô hình có thể dự đoán Severity cho lỗi mới dựa trên hai thuộc tính đầu vào là Bug_Count và Crash.

3.5. Lựa chọn thuật toán

Đề tài sử dụng thuật toán Decision Tree, hay còn gọi là cây quyết định, để xây dựng mô hình phân loại mức độ nghiêm trọng của lỗi phần mềm.

Decision Tree là một thuật toán thuộc nhóm học có giám sát. Thuật toán này hoạt động bằng cách tạo ra các nút quyết định dựa trên giá trị của dữ liệu đầu vào. Mỗi nút trong cây là một điều kiện kiểm tra. Dựa vào kết quả của điều kiện đó, dữ liệu sẽ đi theo các nhánh khác nhau cho đến khi mô hình đưa ra kết quả cuối cùng.

Trong bài toán này, Decision Tree sử dụng hai thuộc tính Bug_Count và Crash để dự đoán Severity.

Có thể hiểu cách hoạt động của cây quyết định như sau:

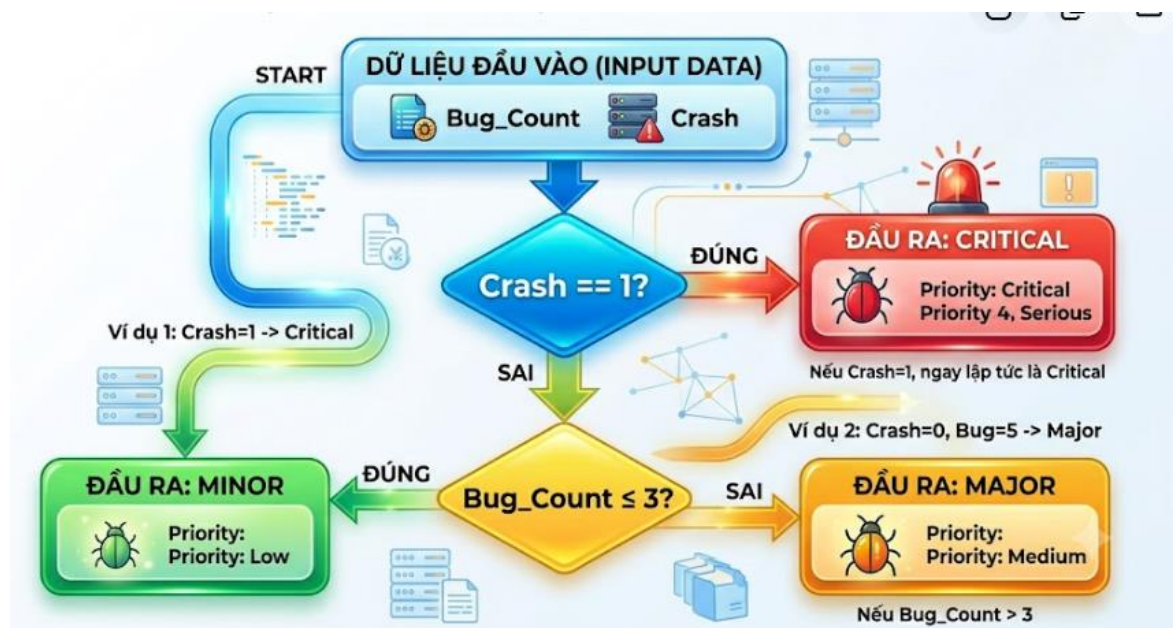
- Nếu Crash = 1 thì lỗi có khả năng thuộc nhóm Critical.
- Nếu Crash = 0 và Bug_Count nhỏ thì lỗi có thể thuộc nhóm Minor.
- Nếu Crash = 0 và Bug_Count lớn hơn thì lỗi có thể thuộc nhóm Major.

Thuật toán Decision Tree được lựa chọn trong đề tài vì các lý do sau:

- Dễ triển khai bằng ngôn ngữ Python.
- Phù hợp với bài toán phân loại.
- Dễ hiểu và dễ giải thích kết quả.
- Phù hợp với bộ dữ liệu nhỏ trong đề tài.

- Có thể minh họa dưới dạng cây quyết định trực quan.

Trong phạm vi đề tài, Decision Tree là thuật toán phù hợp vì mục tiêu chính là mô phỏng cách Machine Learning hỗ trợ QA phân loại mức độ nghiêm trọng của lỗi phần mềm.



Hình 3.5 Mô hình cây quyết định (Decision Tree)

3.6. Xây dựng mô hình Machine Learning

Quá trình xây dựng mô hình Machine Learning trong đề tài được thực hiện bằng ngôn ngữ lập trình Python và thư viện Scikit-learn. Các bước thực hiện gồm đọc dữ liệu, tách dữ liệu, chia tập huấn luyện và kiểm tra, huấn luyện mô hình, đánh giá và dự đoán kết quả.

Bước 1: Đọc dữ liệu

Đầu tiên, chương trình sử dụng thư viện Pandas để đọc dữ liệu từ file bugs.csv.

Dữ liệu sau khi đọc gồm ba cột:

- Bug_Count
- Crash
- Severity

Việc đọc dữ liệu giúp chương trình lấy thông tin từ file CSV và đưa vào quá trình xử lý

Bước 2: Tách dữ liệu đầu vào đầu ra

Sau khi đọc dữ liệu, chương trình tách dữ liệu thành hai phần:

Dữ liệu đầu vào X gồm:

- Bug_Count
- Crash

Dữ liệu đầu ra y gồm:

- Severity

Trong đó, X là dữ liệu dùng để mô hình học, còn y là nhãn mà mô hình cần dự đoán.

Bước 3: Chia dữ liệu

Dữ liệu được chia thành hai phần:

- Tập huấn luyện (Training Set): dùng để huấn luyện mô hình.
- Tập kiểm tra (Testing Set): dùng để đánh giá khả năng dự đoán của mô hình.

Việc chia dữ liệu giúp kiểm tra xem mô hình có thể dự đoán đúng trên dữ liệu chưa được học hay không

Bước 4: Huấn luyện mô hình

Mô hình được huấn luyện bằng thuật toán Decision Tree. Quá trình huấn luyện được thực hiện bằng lệnh:

```
model.fit(X_train, y_train)
```

Lệnh này giúp mô hình học mối quan hệ giữa dữ liệu đầu vào Bug_Count, Crash và nhãn đầu ra Severity.

Bước 5: Dự đoán kết quả

Sau khi mô hình được huấn luyện, chương trình sử dụng lệnh:

```
model.predict()
```

để dự đoán mức độ nghiêm trọng của lỗi mới.

Ví dụ, khi đưa vào mô hình dữ liệu mới:

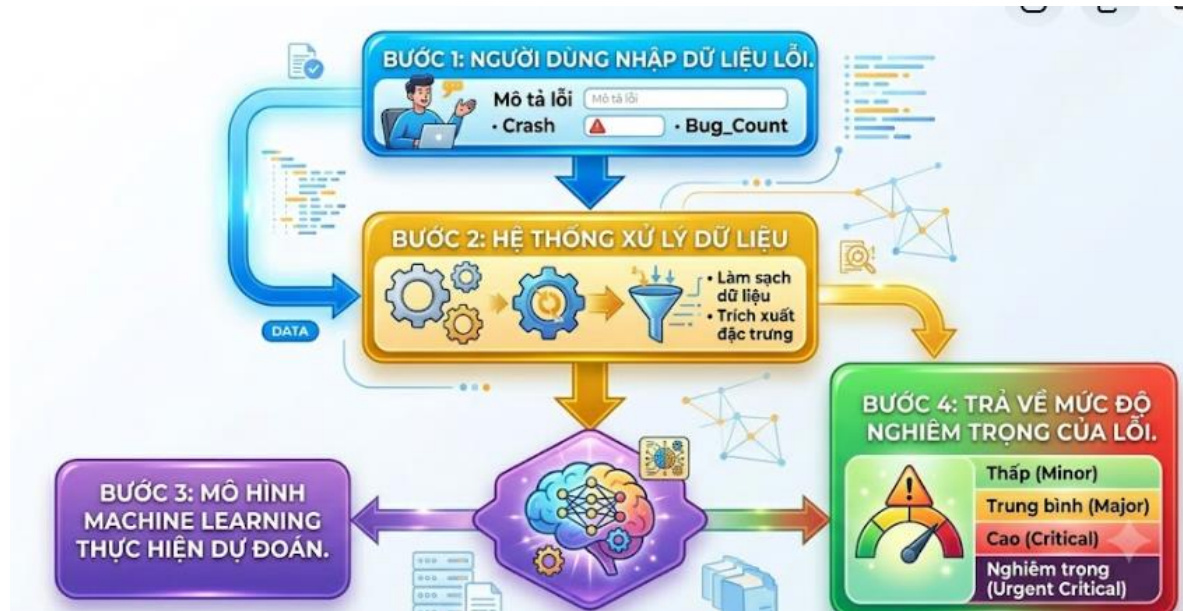
Bug_Count=8

Crash = 1

mô hình dự đoán kết quả là:

Critical - Lỗi nghiêm trọng.

Kết quả này phù hợp với ý nghĩa thực tế, vì lỗi có số lượng lớn và làm hệ thống bị sập thường được xem là lỗi nghiêm trọng.



Hình 3.6: Sơ đồ hoạt động của hệ thống

3.7. Quy trình hoạt động của hệ thống

Quy trình hoạt động của hệ thống phân loại lỗi phần mềm được mô tả như sau:

Bước 1: Người dùng chuẩn bị dữ liệu lỗi phần mềm.

Dữ liệu gồm số lượng lỗi phát hiện và trạng thái hệ thống có bị sập hay không.

Bước 2: Hệ thống đọc dữ liệu từ file bugs.csv.

Dữ liệu được đưa vào chương trình Python để xử lý.

Bước 3: Hệ thống tách dữ liệu thành đầu vào và đầu ra.

Dữ liệu đầu vào gồm Bug_Count và Crash. Dữ liệu đầu ra là Severity.

Bước 4: Mô hình Decision Tree được huấn luyện.

Mô hình học từ dữ liệu đã có sẵn nhãn để nhận biết quy luật phân loại lỗi.

Bước 5: Mô hình thực hiện dự đoán lỗi mới.

Khi có dữ liệu lỗi mới, mô hình dự đoán lỗi thuộc nhóm Minor, Major hoặc

Critical.

Bước 6: Hệ thống trả về kết quả phân loại.

Kết quả dự đoán giúp QA xác định lỗi thuộc mức độ nào và có thể ưu tiên xử lý lỗi phù hợp.

Quy trình này cho thấy cách áp dụng lý thuyết Machine Learning vào một bài toán cụ thể trong kiểm thử phần mềm.

Tóm tắt chương 3

Chương 3 đã trình bày bài toán phân loại mức độ nghiêm trọng của lỗi phần mềm bằng Machine Learning. Nội dung chương đã mô tả rõ dữ liệu đầu vào gồm Bug_Count và Crash, dữ liệu đầu ra là Severity, đồng thời phân tích ý nghĩa của từng thuộc tính trong bộ dữ liệu.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Môi trường thực nghiệm

Sau khi phân tích bài toán và lựa chọn thuật toán Decision Tree ở Chương 3, chương này tiến hành thực nghiệm xây dựng mô hình Machine Learning để phân loại mức độ nghiêm trọng của lỗi phần mềm.

Mục đích của phần thực nghiệm là kiểm tra khả năng áp dụng thuật toán Decision Tree vào bài toán phân loại lỗi dựa trên dữ liệu mẫu. Mô hình sẽ học từ bộ dữ liệu bugs.csv, sau đó dự đoán mức độ nghiêm trọng của lỗi mới dựa trên hai thuộc tính đầu vào là Bug_Count và Crash.

Môi trường thực nghiệm được sử dụng trong đề tài gồm:

Thành phần	Nội dung
Hệ điều hành	Windows
Ngôn ngữ lập trình	Python 3.9
Môi trường lập trình	Visual Studio Code
Thư viện xử lý dữ liệu	Pandas
Thư viện học máy	Scikit-learn
Thuật toán sử dụng	Decision Tree
File dữ liệu	bugs.csv

Các thư viện chính được sử dụng trong chương trình gồm:

- pandas: dùng để đọc dữ liệu từ file bugs.csv.

- `train_test_split`: dùng để chia dữ liệu thành tập huấn luyện và tập kiểm tra.
- `DecisionTreeClassifier`: dùng để xây dựng mô hình cây quyết định.
- `accuracy_score`: dùng để đánh giá độ chính xác của mô hình.

Việc sử dụng Python và thư viện Scikit-learn giúp quá trình xây dựng mô hình Machine Learning trở nên đơn giản, dễ triển khai và phù hợp với bài toán thực nghiệm trong đề tài.

4.2. Cài đặt thư viện cần thiết

Trước khi xây dựng mô hình, cần cài đặt các thư viện cần thiết cho chương trình. Trong đề tài này, thư viện quan trọng nhất là Scikit-learn vì thư viện này cung cấp sẵn thuật toán Decision Tree và các hàm hỗ trợ chia dữ liệu, đánh giá mô hình.

Lệnh cài đặt thư viện:

```
pip install scikit-learn
```

Ngoài ra, thư viện Pandas cũng được sử dụng để đọc và xử lý dữ liệu từ file CSV.

Lệnh cài đặt Pandas:

```
pip install pandas
```

Sau khi cài đặt thành công, chương trình có thể import các thư viện cần thiết và bắt đầu quá trình thực nghiệm.

```
Call --upgrade pip command.
PS C:\Users\FPT\Downloads\KTPM_CUOIKY> c:\Users\FPT\Downloads\KTPM_CUOIKY\.venv\Scripts\python.exe -m pip instal
l scikit-learn
Collecting scikit-learn
  Using cached scikit_learn-1.6.1-cp39-cp39-win_amd64.whl.metadata (15 kB)
Requirement already satisfied: numpy>=1.19.5 in .\venv\lib\site-packages (from scikit-learn) (2.0.2)
Collecting scipy>=1.6.0 (from scikit-learn)
  Using cached scipy-1.13.1-cp39-cp39-win_amd64.whl.metadata (60 kB)
Collecting joblib>=1.2.0 (from scikit-learn)
  Using cached joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.1.0 (from scikit-learn)
  Using cached threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Using cached scikit_learn-1.6.1-cp39-cp39-win_amd64.whl (11.2 MB)
Using cached joblib-1.5.3-py3-none-any.whl (309 kB)
Using cached scipy-1.13.1-cp39-cp39-win_amd64.whl (46.2 MB)
Using cached threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Installing collected packages: threadpoolctl, scipy, joblib, scikit-learn
Successfully installed joblib-1.5.3 scikit-learn-1.6.1 scipy-1.13.1 threadpoolctl-3.6.0
PS C:\Users\FPT\Downloads\KTPM_CUOIKY> □
```

Hình 4.1. Cài đặt thư viện Scikit-learn

Nhận xét

Việc cài đặt thư viện thành công là bước quan trọng để chương trình có thể sử dụng các công cụ hỗ trợ Machine Learning. Trong bài thực nghiệm, Scikit-learn được sử dụng để tạo mô hình Decision Tree, còn Pandas được sử dụng để đọc dữ liệu lỗi phần mềm từ file bugs.csv

4.2. Chuẩn bị dữ liệu thực nghiệm

Bộ dữ liệu thực nghiệm được lưu trong file bugs.csv. Đây là file dữ liệu dạng CSV, gồm các mẫu lỗi phần mềm được xây dựng để phục vụ cho quá trình huấn luyện và kiểm tra mô hình.

Cấu trúc dữ liệu gồm ba cột:

Thuộc tính	Ý nghĩa
Bug_Count	Số lượng lỗi được phát hiện
Crash	Trạng thái hệ thống có bị sập hay không
Severity	Mức độ nghiêm trọng của lỗi

Trong đó:

- Bug_Count là dữ liệu đầu vào, thể hiện số lượng lỗi phát hiện.
- Crash là dữ liệu đầu vào, thể hiện lỗi có làm hệ thống bị sập hay không.
- Severity là nhãn dữ liệu, thể hiện mức độ nghiêm trọng của lỗi.

Một phần dữ liệu trong file bugs.csv như sau:

Bug_Count	Crash	Severity
1	0	Minor
2	0	Minor
3	0	Minor
4	0	Major
5	0	Major
6	1	Critical

7	1	Critical
8	1	Critical

Dữ liệu này được sử dụng để mô hình học mối quan hệ giữa số lượng lỗi, trạng thái hệ thống bị sập và mức độ nghiêm trọng của lỗi.

Đoạn lệnh đọc dữ liệu:

```
import pandas as pd
df = pd.read_csv("bugs.csv")
print("Dữ liệu lỗi:")
print(df)
```

Giải thích:

- `import pandas as pd`: khai báo sử dụng thư viện Pandas.
- `pd.read_csv("bugs.csv")`: đọc dữ liệu từ file bugs.csv.
- `df`: biến lưu toàn bộ dữ liệu sau khi đọc.
- `print(df)`: hiển thị dữ liệu ra màn hình để kiểm tra

```
PS C:\Users\FPT\Downloads\KTPM_CUOIKY> python bug_ai.py
Dữ liệu lỗi:
  Bug_Count  Crash  Severity
0          1      0   Minor
1          2      0   Minor
2          3      0   Minor
3          4      0   Major
4          5      0   Major
5          6      1  Critical
6          7      1  Critical
7          8      1  Critical
8          9      1  Critical
9         10      1  Critical
10          3      1   Major
11          4      1   Major
12          5      1  Critical
13          2      0   Minor
14          7      0   Major
15          8      1  Critical
```

Hình 4.2. Kết quả đọc dữ liệu từ file bugs.csv

Nhận xét

Kết quả đọc dữ liệu cho thấy chương trình đã nạp thành công file bugs.csv. Dữ liệu có đủ ba thuộc tính cần thiết là Bug_Count, Crash và Severity. Đây là cơ sở để tiến hành tách dữ liệu và huấn luyện mô hình Machine Learning.

4.3. Tách dữ liệu đầu vào và đầu ra

Sau khi đọc dữ liệu, chương trình tiến hành tách dữ liệu thành hai phần: dữ liệu đầu vào và dữ liệu đầu ra.

Dữ liệu đầu vào được ký hiệu là X, gồm hai thuộc tính:

- Bug_Count
- Crash

Dữ liệu đầu ra được ký hiệu là y, gồm thuộc tính:

- Severity

Đoạn lệnh thực hiện:

```
X = df[['Bug_Count', 'Crash']]
```

```
y = df['Severity']
```

Giải thích:

- `X = df[['Bug_Count', 'Crash']]`: lấy hai cột Bug_Count và Crash làm dữ liệu đầu vào cho mô hình.
- `y = df['Severity']`: lấy cột Severity làm nhãn dữ liệu cần dự đoán.

Trong bài toán này, mô hình sẽ học từ X để dự đoán y. Nói cách khác, mô hình dựa vào số lượng lỗi và trạng thái hệ thống có bị sập hay không để phân loại lỗi thành Minor, Major hoặc Critical.

Việc tách dữ liệu như trên phù hợp với phương pháp học có giám sát, vì dữ liệu đã có sẵn nhãn kết quả là Severity.

4.4. Chia dữ liệu huấn luyện và kiểm tra

Để đánh giá khả năng dự đoán của mô hình, dữ liệu được chia thành hai phần:

Tập huấn luyện: dùng để mô hình học.

Tập kiểm tra: dùng để kiểm tra độ chính xác của mô hình.

Đoạn lệnh thực hiện:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Giải thích:

`train_test_split`: hàm dùng để chia dữ liệu thành tập huấn luyện và tập kiểm tra.
`test_size=0.2`: 20% dữ liệu được dùng để kiểm tra, 80% dữ liệu dùng để huấn luyện.

`random_state=42`: giúp kết quả chia dữ liệu ổn định khi chạy lại chương trình.

`X_train, y_train`: dữ liệu dùng để huấn luyện mô hình.

`X_test, y_test`: dữ liệu dùng để kiểm tra mô hình.

Việc chia dữ liệu giúp đánh giá mô hình khách quan hơn. Nếu chỉ huấn luyện và kiểm tra trên cùng một dữ liệu thì kết quả có thể không phản ánh đúng khả năng dự đoán của mô hình đối với dữ liệu mới

4.5. Huấn luyện mô hình Decision Tree

Sau khi chia dữ liệu, chương trình tiến hành xây dựng và huấn luyện mô hình Decision Tree.

Đoạn lệnh thực hiện:

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

Giải thích:

- `DecisionTreeClassifier()`: tạo mô hình cây quyết định dùng cho bài toán phân loại.
- `model`: biến lưu mô hình Machine Learning.
- `model.fit(X_train, y_train)`: huấn luyện mô hình bằng dữ liệu huấn luyện.

Trong quá trình huấn luyện, mô hình học mối quan hệ giữa hai thuộc tính đầu vào là `Bug_Count`, `Crash` và nhãn đầu ra là `Severity`.

Có thể hiểu đơn giản quá trình học như sau:

- Nếu `Bug_Count` thấp và `Crash` = 0, lỗi thường thuộc nhóm Minor.
- Nếu `Bug_Count` cao hơn nhưng `Crash` = 0, lỗi có thể thuộc nhóm Major.
- Nếu `Crash` = 1, lỗi có khả năng thuộc nhóm Critical.

Sau khi huấn luyện, mô hình có thể sử dụng các quy luật đã học để dự đoán mức độ nghiêm trọng của lỗi mới.

```
)  
  
# Tạo mô hình AI  
model = DecisionTreeClassifier()  
  
# Huấn luyện  
model.fit(X_train, y_train)  
  
# Kiểm tra độ chính xác  
y_pred = model.predict(X_test)  
  
acc = accuracy_score(y_test, y_pred)  
  
print("\nĐộ chính xác:", round(acc * 100, 2), "%")
```

Hình 4.5. Quá trình huấn luyện mô hình Decision Tree

Nhận xét

Mô hình Decision Tree phù hợp với bài toán thực nghiệm vì dữ liệu có kích thước nhỏ, các thuộc tính đầu vào rõ ràng và kết quả đầu ra thuộc dạng phân loại. Ngoài ra, thuật toán này có cách hoạt động dễ hiểu, phù hợp để trình bày trong báo cáo môn học.

4.6. Đánh giá độ chính xác của mô hình

Sau khi huấn luyện, mô hình được sử dụng để dự đoán kết quả trên tập kiểm tra. Kết quả dự đoán được so sánh với nhãn thực tế để tính độ chính xác.

Đoạn lệnh thực hiện:

```
from sklearn.metrics import accuracy_score  
y_pred = model.predict(X_test)  
acc = accuracy_score(y_test, y_pred)  
print("Độ chính xác:", round(acc * 100, 2), "%")
```

Giải thích:

- `model.predict(X_test)`: dự đoán nhãn Severity cho dữ liệu kiểm tra.
- `accuracy_score(y_test, y_pred)`: tính tỷ lệ dự đoán đúng của mô hình.

- `round(acc * 100, 2)`: chuyển kết quả sang phần trăm và làm tròn hai chữ số thập phân.

Kết quả thực nghiệm:

Độ chính xác: 75.0 %

Điều này có nghĩa là mô hình dự đoán đúng khoảng 75% dữ liệu kiểm tra.

```

PS C:\Users\FPT\Downloads\KTPM_CUOIKY> python bug_ai.py
Dữ liệu lỗi:
  Bug_Count  Crash  Severity
0          1      0    Minor
1          2      0    Minor
2          3      0    Minor
3          4      0    Major
4          5      0    Major
5          6      1  Critical
6          7      1  Critical
7          8      1  Critical
8          9      1  Critical
9         10      1  Critical
10         3      1    Major
11         4      1    Major
12         5      1  Critical
13         2      0    Minor
14         7      0    Major
15         8      1  Critical

Độ chính xác: 75.0 %

```

Hình 4.6. Kết quả đánh giá độ chính xác của mô hình

Nhận xét:

Kết quả Accuracy = 75% cho thấy mô hình đã học được một số quy luật cơ bản từ dữ liệu. Tuy nhiên, do bộ dữ liệu chỉ gồm 16 mẫu nên độ chính xác chưa đạt mức tối ưu. Nếu có thêm nhiều dữ liệu lỗi thực tế hơn, mô hình có thể học tốt hơn và cải thiện độ chính xác

4.7. Thử nghiệm dự đoán lỗi mới

Sau khi đánh giá mô hình, chương trình tiếp tục thử nghiệm dự đoán một lỗi mới. Dữ liệu lỗi mới được đưa vào mô hình gồm:

Bug_Count Crash

8 1

Trong đó:

- Bug_Count = 8: số lượng lỗi phát hiện là 8.
- Crash = 1: lỗi làm hệ thống bị sập.

Đoạn lệnh thực hiện:

```
new_bug = pd.DataFrame([[8, 1]], columns=['Bug_Count', 'Crash'])
result = model.predict(new_bug)
```

```
print("Bug mới:")
print("Bug_Count = 8")
print("Crash = 1")
```

```
print("AI dự đoán:")
if result[0] == "Minor":
    print("Minor - Lỗi nhẹ")
elif result[0] == "Major":
    print("Major - Lỗi trung bình")
else:
    print("Critical - Lỗi nghiêm trọng")
```

Giải thích:

- new_bug: dữ liệu lỗi mới cần dự đoán.
- pd.DataFrame([[8, 1]], columns=['Bug_Count', 'Crash']): tạo dữ liệu mới có cùng tên cột với dữ liệu huấn luyện.
- model.predict(new_bug): mô hình dự đoán mức độ nghiêm trọng của lỗi mới.
- Câu lệnh if elif else: dùng để hiển thị kết quả dự đoán bằng tiếng Việt.

Kết quả chương trình:

Bug mới:

Bug_Count = 8

Crash = 1

AI dự đoán:

Critical - Lỗi nghiêm trọng

```
15      8      1 Critical

Độ chính xác: 75.0 %

Bug mới:
Bug_Count = 8
Crash = 1

AI dự đoán:
Critical - Lỗi nghiêm trọng
PS C:\Users\FPT\Downloads\KTPM_CUOIKY>
```

Hình 4.7. Kết quả dự đoán mức độ nghiêm trọng của lỗi mới

4.8. Đánh giá kết quả thực nghiệm

4.8.1. Kết quả đạt được

Qua quá trình thực nghiệm, đề tài đã đạt được một số kết quả sau:

- Đọc được dữ liệu lỗi phần mềm từ file bugs.csv.
- Tách được dữ liệu đầu vào và nhãn đầu ra.
- Chia được dữ liệu thành tập huấn luyện và tập kiểm tra.
- Xây dựng được mô hình Decision Tree bằng thư viện Scikit-learn.
- Huấn luyện được mô hình phân loại lỗi phần mềm.
- Đánh giá được mô hình bằng chỉ số Accuracy.
- Thử nghiệm dự đoán lỗi mới thành công.

Kết quả thực nghiệm cho thấy mô hình đạt độ chính xác 75% và dự đoán lỗi mới có Bug_Count = 8, Crash = 1 thuộc nhóm Critical.

4.8.2. Ưu điểm của mô hình

Mô hình Decision Tree trong đề tài có một số ưu điểm:

- Dễ xây dựng và triển khai bằng Python.
- Phù hợp với bài toán phân loại mức độ nghiêm trọng của lỗi.
- Dễ hiểu và dễ giải thích kết quả.
- Có thể hỗ trợ QA xác định lỗi nào cần ưu tiên xử lý.
- Phù hợp với mục tiêu mô phỏng ứng dụng AI/ML trong kiểm thử phần mềm.

Với mô hình này, thay vì đánh giá lỗi hoàn toàn thủ công, QA có thể sử dụng dữ liệu đầu vào để hệ thống hỗ trợ phân loại mức độ nghiêm trọng của lỗi.

4.8.3. Hạn chế của mô hình

Bên cạnh các ưu điểm, mô hình vẫn còn một số hạn chế:

- Bộ dữ liệu còn nhỏ, chỉ gồm 16 mẫu.
- Dữ liệu là dữ liệu mẫu, chưa phải dữ liệu thực tế từ dự án phần mềm.
- Mô hình chỉ sử dụng hai thuộc tính đầu vào là Bug_Count và Crash.
- Độ chính xác 75% chưa phải là kết quả cao.
- Chưa so sánh với các thuật toán Machine Learning khác.
- Chưa xây dựng giao diện nhập lỗi và hiển thị kết quả trực quan.

Những hạn chế này cho thấy mô hình hiện tại phù hợp với mục đích học tập và mô phỏng, nhưng nếu muốn áp dụng vào thực tế cần tiếp tục mở rộng và cải tiến.

4.8.4. Khả năng ứng dụng trong kiểm thử phần mềm

Mặc dù mô hình còn đơn giản, kết quả thực nghiệm cho thấy Machine Learning có thể hỗ trợ hoạt động kiểm thử phần mềm, đặc biệt trong việc phân loại mức độ nghiêm trọng của lỗi.

Trong thực tế, nếu có dữ liệu lỗi đầy đủ hơn, mô hình có thể được dùng để:

- Hỗ trợ QA đánh giá mức độ nghiêm trọng của lỗi.
- Ưu tiên xử lý lỗi Critical trước.
- Giảm thời gian phân loại lỗi thủ công.
- Hỗ trợ lập trình viên tập trung xử lý các lỗi ảnh hưởng lớn đến hệ thống.
- Góp phần nâng cao chất lượng phần mềm trước khi triển khai.

Như vậy, việc ứng dụng AI/ML trong kiểm thử phần mềm có thể giúp tối ưu quy trình QA và nâng cao hiệu quả đảm bảo chất lượng sản phẩm.

Tóm tắt chương 4

Chương 4 đã trình bày quá trình thực nghiệm xây dựng mô hình Machine Learning bằng thuật toán Decision Tree để phân loại mức độ nghiêm trọng của lỗi phần mềm. Kết quả thực nghiệm cho thấy mô hình đạt độ chính xác 75% và có khả năng dự đoán lỗi mới thuộc nhóm Critical khi đầu vào là Bug_Count = 8 và Crash = 1. Điều này chứng minh khả năng ứng dụng của Machine

Learning trong việc hỗ trợ hoạt động kiểm thử và đảm bảo chất lượng phần mềm.

TÀI LIỆU THAM KHẢO

1. Scikit-learn Developers, Scikit-learn Documentation: `DecisionTreeClassifier`, Scikit-learn Official Documentation.
2. Scikit-learn Developers, Scikit-learn Documentation: `train_test_split`, Scikit-learn Official Documentation.
3. Pandas Development Team, Pandas Documentation: `pandas.read_csv`, Pandas Official Documentation.
4. Python Software Foundation, Python Documentation, Python Official Documentation.

.